

SP26 CS570 Week 1 - Lab Notebook Spark Intro

January 29, 2026

1 CS570 – Week 1 Lab: Spark First Contact

Notebook: week1_spark_intro.ipynb

1.1 Goal

Understand **how Spark behaves as a system**: - What is *planned* vs what is *executed* - Transformations vs actions - Lazy evaluation and DAGs

This is **not** a performance lab. We are not tuning Spark or benchmarking runtimes today.

1.2 0. Setup check (run this first)

This cell confirms your environment is roughly correct.

```
[1]: import sys
import pyspark

print("Python:", sys.version.split()[0])
print("PySpark:", pyspark.__version__)
```

Python: 3.12.3

PySpark: 3.5.3

1.3 1. Create a SparkSession

This is your entry point into Spark from Python.

```
[3]: from pyspark.sql import SparkSession

spark = (
    SparkSession.builder
    .appName("cs570-week1")
    .master("local[*]")    # use all local CPU cores
    .getOrCreate()
)

spark
```

```
[3]: <pyspark.sql.session.SparkSession at 0x7f502af3c3b0>
```

1.3.1 Optional: Find the Spark UI link

If the UI is enabled, this prints a URL you can open in your browser.

```
[4]: ui_url = spark.sparkContext.uiWebUrl
print("Spark UI:", ui_url if ui_url else "(Spark UI not available in this_
environment)")
```

Spark UI: http://10.255.255.254:4040

1.3.2 Configure display settings

```
[5]: spark.conf.set("spark.sql.repl.eagerEval.enabled", True)
spark.conf.set("spark.sql.repl.eagerEval.maxNumRows", 5)
spark.conf.set("spark.sql.debug.maxToStringFields", 100)

# Optional: make pandas previews show all columns too
import pandas as pd
pd.set_option("display.max_columns", None)
pd.set_option("display.width", 200)
pd.set_option("display.max_colwidth", None)
```

1.4 2. Get a dataset

1.4.1 Generate a small synthetic event dataset

This avoids download friction and still matches the lecture narrative (events, clicks, logs).

```
[6]: from pathlib import Path
import pandas as pd
import random
import datetime as dt

# Folder where we'll store data next to this notebook
data_dir = Path("data")
data_dir.mkdir(exist_ok=True)

# Synthetic clickstream-like events
random.seed(570)

users = [f"u{n:03d}" for n in range(1, 61)]
pages = ["/", "/search", "/product", "/cart", "/checkout", "/help", "/profile"]
devices = ["mobile", "desktop", "tablet"]
countries = ["US", "DE", "IN", "BR", "CA", "GB"]

rows = []
start = dt.datetime(2026, 1, 1, 9, 0, 0)
```

```

for i in range(1200):
    t = start + dt.timedelta(seconds=30*i)
    rows.append({
        "event_time": t.isoformat(),
        "user_id": random.choice(users),
        "page": random.choice(pages),
        "device": random.choice(devices),
        "country": random.choice(countries),
        "dwell_seconds": random.randint(1, 120),
        "is_purchase": 1 if random.random() < 0.04 else 0
    })

pdf = pd.DataFrame(rows)

DATA_PATH = data_dir / "week1_events.csv"
pdf.to_csv(DATA_PATH, index=False)

DATA_PATH.as_posix(), pdf.head()

```

```

[6]: ('data/week1_events.csv',
      event_time user_id      page  device country dwell_seconds
is_purchase
0  2026-01-01T09:00:00    u005  /search  mobile      IN              7
1
1  2026-01-01T09:00:30    u052  /product  desktop    CA             76
0
2  2026-01-01T09:01:00    u024  /product  mobile     BR            102
0
3  2026-01-01T09:01:30    u021  /product  tablet     US             92
0
4  2026-01-01T09:02:00    u029  /product  tablet     GB             68
1)

```

1.5 3. Load data into Spark

Key idea: **loading + defining** is not the same as **executing**.

```

[7]: from pyspark.sql.types import *

df = (
    spark.read
    .option("header", True)
    .option("inferSchema", True)
    .csv(DATA_PATH.as_posix())
)

df

```

```
[7]: +-----+-----+-----+-----+-----+-----+-----+
|      event_time|user_id|   page| device|country|dwell_seconds|is_purchase|
+-----+-----+-----+-----+-----+-----+-----+
|2026-01-01 09:00:00|  u005| /search| mobile|   IN|         7|         1|
|2026-01-01 09:00:30|  u052|/product|desktop|   CA|        76|         0|
|2026-01-01 09:01:00|  u024|/product| mobile|   BR|       102|         0|
|2026-01-01 09:01:30|  u021|/product| tablet|   US|        92|         0|
|2026-01-01 09:02:00|  u029|/product| tablet|   GB|        68|         1|
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

```
[8]: df.printSchema()
```

```
root
 |-- event_time: timestamp (nullable = true)
 |-- user_id: string (nullable = true)
 |-- page: string (nullable = true)
 |-- device: string (nullable = true)
 |-- country: string (nullable = true)
 |-- dwell_seconds: integer (nullable = true)
 |-- is_purchase: integer (nullable = true)
```

```
[9]: df.show(5, truncate=False)
```

```
+-----+-----+-----+-----+-----+-----+-----+
|event_time      |user_id|page   |device |country|dwell_seconds|is_purchase|
+-----+-----+-----+-----+-----+-----+-----+
|2026-01-01 09:00:00|u005   |/search|mobile |IN     |7           |1           |
|2026-01-01 09:00:30|u052   |/product|desktop|CA     |76          |0           |
|2026-01-01 09:01:00|u024   |/product|mobile |BR     |102         |0           |
|2026-01-01 09:01:30|u021   |/product|tablet |US     |92          |0           |
|2026-01-01 09:02:00|u029   |/product|tablet |GB     |68          |1           |
+-----+-----+-----+-----+-----+-----+-----+
only showing top 5 rows
```

1.6 4. Transformations (define work, don't execute yet)

Try a few transformations. These should feel “instant” because Spark is *building a plan*.

```
[10]: from pyspark.sql import functions as F

# Example: filter + select + derived column
df_t = (
    df
    .filter(F.col("country") == "US")
```

```

        .select("event_time", "user_id", "page", "device", "country",
        ↪ "dwell_seconds", "is_purchase")
        .withColumn("is_high_intent", (F.col("page").isin(["/cart", "/checkout"])).
        ↪ cast("int"))
    )

df_t

```

```

[10]: +-----+-----+-----+-----+-----+-----+-----+
+-----+
|          event_time|user_id|    page|
device|country|dwell_seconds|is_purchase|is_high_intent|
+-----+-----+-----+-----+-----+-----+-----+
+-----+
|2026-01-01 09:01:30|   u021| /product| tablet|    US|          92|
0|                0|
|2026-01-01 09:04:30|   u052|  /cart|desktop|    US|          5|
0|                1|
|2026-01-01 09:06:30|   u015|  /cart| mobile|    US|         47|
0|                1|
|2026-01-01 09:12:00|   u043| /profile|desktop|    US|         98|
0|                0|
|2026-01-01 09:13:00|   u048|/checkout| tablet|    US|         29|
0|                1|
+-----+-----+-----+-----+-----+-----+-----+
+-----+
only showing top 5 rows

```

1.6.1 Inspect the logical plan (no execution yet)

This shows the **plan** Spark has built.

```

[11]: df_t.explain(True)

== Parsed Logical Plan ==
'Project [event_time#17, user_id#18, page#19, device#20, country#21,
dwell_seconds#22, is_purchase#23, cast('page IN (/cart,/checkout) as int) AS
is_high_intent#147]
+- Project [event_time#17, user_id#18, page#19, device#20, country#21,
dwell_seconds#22, is_purchase#23]
   +- Filter (country#21 = US)
      +- Relation [event_time#17,user_id#18,page#19,device#20,country#21,dwell_s
econds#22,is_purchase#23] csv

== Analyzed Logical Plan ==
event_time: timestamp, user_id: string, page: string, device: string, country:
string, dwell_seconds: int, is_purchase: int, is_high_intent: int
Project [event_time#17, user_id#18, page#19, device#20, country#21,

```

```

dwell_seconds#22, is_purchase#23, cast(page#19 IN (/cart,/checkout) as int) AS
is_high_intent#147]
+- Project [event_time#17, user_id#18, page#19, device#20, country#21,
dwell_seconds#22, is_purchase#23]
  +- Filter (country#21 = US)
    +- Relation [event_time#17,user_id#18,page#19,device#20,country#21,dwell_s
econds#22,is_purchase#23] csv

== Optimized Logical Plan ==
Project [event_time#17, user_id#18, page#19, device#20, country#21,
dwell_seconds#22, is_purchase#23, cast(page#19 IN (/cart,/checkout) as int) AS
is_high_intent#147]
+- Filter (isnotnull(country#21) AND (country#21 = US))
  +- Relation [event_time#17,user_id#18,page#19,device#20,country#21,dwell_seco
nds#22,is_purchase#23] csv

== Physical Plan ==
*(1) Project [event_time#17, user_id#18, page#19, device#20, country#21,
dwell_seconds#22, is_purchase#23, cast(page#19 IN (/cart,/checkout) as int) AS
is_high_intent#147]
+- *(1) Filter (isnotnull(country#21) AND (country#21 = US))
  +- FileScan csv [event_time#17,user_id#18,page#19,device#20,country#21,dwell_
seconds#22,is_purchase#23] Batched: false, DataFilters: [isnotnull(country#21),
(country#21 = US)], Format: CSV, Location: InMemoryFileIndex(1
paths)[file:/home/fsehay/week-1/data/week1_events.csv], PartitionFilters: [],
PushedFilters: [IsNotNull(country), EqualTo(country,US)], ReadSchema: struct<eve
nt_time:timestamp,user_id:string,page:string,device:string,country:string,dwell_
seconds...

```

1.7 5. Actions (trigger execution)

Now run an action. This is where Spark *actually does work*.

```
[12]: df_t.count()
```

```
[12]: 204
```

```
[13]: # Simple aggregation (GROUP BY)
df_g = (
    df.groupBy("country")
      .agg(
        F.count("*").alias("num_events"),
        F.sum("is_purchase").alias("num_purchases"),
        F.avg("dwell_seconds").alias("avg_dwell_seconds")
      )
      .orderBy(F.desc("num_events"))
)
```

```
df_g.show(truncate=False)
```

country	num_events	num_purchases	avg_dwell_seconds
GB	210	7	59.642857142857146
US	204	8	57.03921568627451
DE	200	14	60.095
BR	197	11	59.401015228426395
CA	195	8	58.697435897435895
IN	194	5	61.74742268041237

1.8 6. Connect back to lecture

Before we wrap up, think about these questions: 1. What is the difference between a **transformation** and an **action**? 2. What does **lazy evaluation** mean in Spark? 3. What surprised you most about Spark's behavior?

1.9 7. (Optional) Small experiment

Change the filter condition and rerun the action.

Predict what will happen before you run it.

```
[14]: df_t2 = df.filter(F.col("device") == "mobile").select("user_id", "page", "device", "country", "is_purchase")

# Predict: Will this do any work yet?
df_t2.explain()

# Now trigger execution
df_t2.count()

== Physical Plan ==
*(1) Filter (isnotnull(device#20) AND (device#20 = mobile))
+- FileScan csv [user_id#18,page#19,device#20,country#21,is_purchase#23]
   Batched: false, DataFilters: [isnotnull(device#20), (device#20 = mobile)],
   Format: CSV, Location: InMemoryFileIndex(1 paths) [file:/home/fsehay/week-1/data/week1_events.csv], PartitionFilters: [],
   PushedFilters: [IsNotNull(device), EqualTo(device,mobile)], ReadSchema:
   struct<user_id:string,page:string,device:string,country:string,is_purchase:int>
```

```
[14]: 407
```

When did Spark actually execute work? Only when actions were called — `count()`, `show()`, and

`collect()` in Step 4. The transformations in Step 3 returned instantly without touching data. Even `explain()` did not trigger execution.

What caused execution to start? Actions caused it. Actions are operations that need to return a result to the driver or write to storage. Each action triggers the entire chain of transformations leading to the result.

How does this relate to lazy evaluation? Spark records transformations as a DAG without executing them. Only when an action demands a result does Spark run the full pipeline. This enables two benefits: (1) the Catalyst optimizer can rearrange/combine operations (e.g., pushing filters down), and (2) Spark avoids computing intermediate results that are never used.